Amrutvahini College of Engineering, Sangamner
Department of Electronics and Telecommunication Engineering
Experiment No.: 04

Date of Performance:

AIM: - To write VHDL code for FIFO.

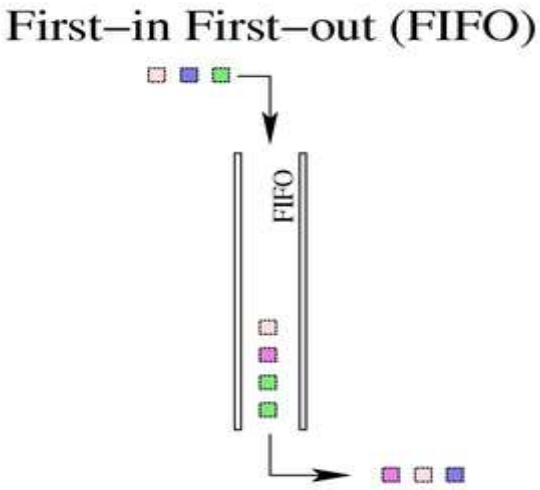
OBJECTIVE: - To study in detail working of FIFO memory.

SOFTWARE: Xilinx 13.4

THEORY: -

FIFO is an acronym for **First In, First Out**, a method for organizing and manipulating a data buffer, where the oldest (first) entry, or 'rear' of the queue, is processed first. It is analogous to processing a queue with first-come, first-served (**FCFS**) behaviour: where the people leave the queue in the order in which they arrive.

FCFS is also the jargon term for the FIFO operating system scheduling algorithm, which gives every process CPU time in the order in which it is demanded. FIFO's opposite is LIFO, Last-In-First-Out, where the youngest entry or 'top of the stack' is processed first. A priority queue is neither FIFO or LIFO but may adopt similar behaviour temporarily or by default. Queueing theory encompasses these methods for processing data structures, as well as interactions between strict-FIFO queues.



Depending on the application, a FIFO could be implemented as a hardware shift register, or using different memory structures, typically a Circular buffer or a kind of List.

Disk scheduling

Disk controllers can use the FIFO as a disk scheduling algorithm to determine the order in which to service disk I/O requests.

Communications and networking

Communications bridges, switches and routers used in Computer networks use FIFOs to hold data packets en route to their next destination. Typically at least one FIFO structure is used per network connection. Some devices feature multiple FIFOs for simultaneously and independently queuing different types of information.

Electronics

FIFOs are commonly used in electronic circuits for buffering and flow control between hardware and software. In its hardware form, a FIFO primarily consists of a set of read and write pointers, storage and control logic. Storage may be SRAM, flip-flops, latches or any other suitable form of storage. For FIFOs of non-trivial size, a dual-port SRAM is usually used, where one port is dedicated to writing and the other to reading.
A synchronous FIFO is a FIFO where the same clock is used for both reading and writing. An asynchronous FIFO uses different clocks for reading and writing. Asynchronous FIFOs introduce metastability issues. A common implementation of an asynchronous FIFO uses a Gray code (or any unit distance code) for the read and write pointers to ensure reliable flag generation. One further note concerning flag generation is that one must necessarily use pointer arithmetic to generate flags for asynchronous FIFO implementations. Conversely, one may use either a "leaky bucket" approach or pointer arithmetic to generate flags in synchronous FIFO implementations.

Examples of FIFO status flags include: full, empty, almost full, almost empty, etc.

The first known FIFO implemented in electronics was done by Peter Alfke in 1969 at Fairchild Semiconductors. Peter Alfke was later a Director at Xilinx.

FIFO full/empty

It has two pointers **front** and **rear**

1. When front equals the rear, the FIFO is empty.
2. When the rear reaches maximum possible value then FIFO is Full.

Conclusion:

Questions:

Q1. Differentiate between signal and variables.
Q2. What is sensitivity parameter (list) for Process? How it affects the operation of Process statement.
Q3. What is the difference between using Process statement in VHDL program and using it in Testbench program?

Reference Book:

Douglas L Perry, "Data Types" in *VHDL Programming by Example*, 4th Edition, McGraw-Hill, New Delhi, India, 2002, ch.4, pp-73-105

VHDL Program:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
entity main is
    Port ( d : in  STD_LOGIC_VECTOR (3 downto 0);
           enq,deq,rst,clk : in  STD_LOGIC;
           q : out  STD_LOGIC_VECTOR (3 downto 0):="0000";
           full,empty : out  STD_LOGIC);
end main;
architecture Behavioral of main is
type memory_type is array (0 to 3) of std_logic_vector(3 downto 0);
signal memory : memory_type :=(others => (others => '0'));
begin
process(clk,rst)
variable front,rear: integer range 0 to 4 :=0;
begin
    if rst='1' then
           front:=0; rear:=0;q<="0000";
    elsif clk'event and clk='1' then
      if enq='1' then
               memory(rear) <= d;
                    rear:=rear+1;
           elsif deq='1' then
                           q<= memory(front);
                           front:=front+1;
           end if;
         if front=rear then
          empty<='1';
                 full<='0';
         elsif front /=4 and rear=4 then
          full<='1';
                 empty<='0';
         else
            full<='0';
           empty<='0';
         end if;
  end if;
end process;
end Behavioral;
```

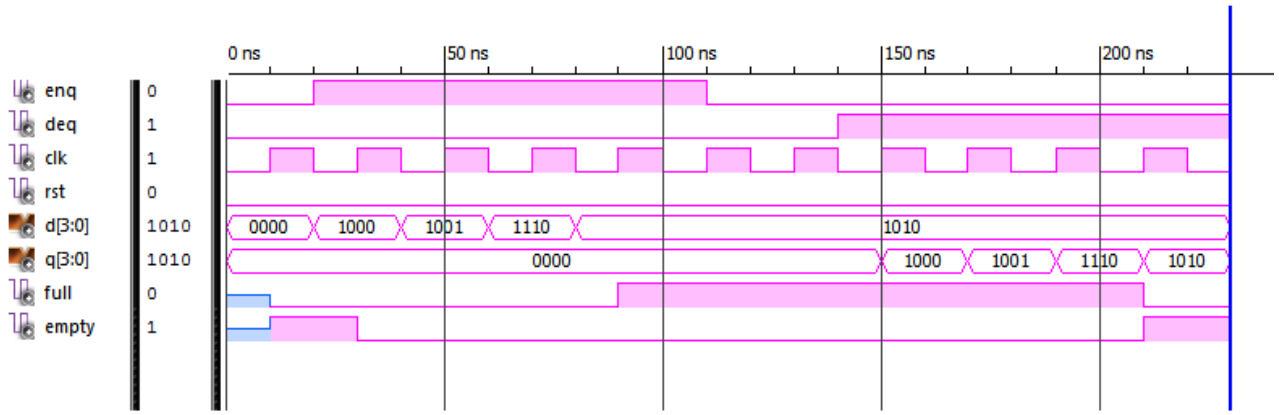
VHDL Testbench:

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.std_logic_unsigned.all;
USE ieee.numeric_std.ALL;
ENTITY te_vhd IS
END te_vhd;
ARCHITECTURE behavior OF te_vhd IS
      COMPONENT main
      PORT(
```

```
            d : IN std_logic_vector(3 downto 0);
            enq : IN std_logic;
            deq,clk : IN std_logic;
            rst : IN std_logic;
            q : OUT std_logic_vector(3 downto 0);
            full : OUT std_logic;
            empty : OUT std_logic
            );
      END COMPONENT;
      SIGNAL enq :  std_logic := '0';
      SIGNAL deq,clk :  std_logic := '0';
      SIGNAL rst :  std_logic := '0';
      SIGNAL d :  std_logic_vector(3 downto 0) := (others=>'0');
      SIGNAL q :  std_logic_vector(3 downto 0);
      SIGNAL full :  std_logic;
      SIGNAL empty :  std_logic;
BEGIN
      uut: main PORT MAP(d => d,enq => enq,deq => deq, clk=>clk, rst=>rst,
            q => q,full => full, empty => empty );

      process
      begin
      wait for 10 ns;
      clk<=not clk;
      end process;

      PROCESS
      BEGIN
      wait for 20 ns;
      d<="1000";
      enq<='1';
      wait for 20 ns;
      d<="1001";
      wait for 20 ns;
      d<="1110";
      wait for 20 ns;
      d<="1010";
      wait for 30 ns;
      enq<='0';
      wait for 30 ns;
      deq<='1';
      wait;
      END PROCESS;
END;
```


0 ns
50 ns
100 ns
150 ns
200 ns
enq
deq
clk
rst
d[3:0]
q[3:0]
full
empty
0
1
1
0
1010
1010
0000
1000
1001
1110
1010
0000
1000
1001
1110
1010